ECE1778: Creative Applications for Mobile Devices

Chen Ying
Assistant Professor, Teaching Stream
Department of Electrical and Computer Engineering
University of Toronto

Previous Lecture

Handling Async Side Effects in Redux

- **Thunks**: Action creators for explicit async logic (`createAsyncThunk`)
- Handle pending, fulfilled, rejected states in extraReducers
- **Middleware**: Automatic side effects (`createListenerMiddleware`)
- Listen for actions and run async work post-reducer

Previous Lecture

Notifications: Enhance user engagement with timely alerts

- **Local Notifications**: Device-scheduled, offline
- Install `expo-notifications` to use Expo Notifications
- Set handler; request permissions; schedule; listen to user taps
- **Push Notifications**: Server-delivered, internet-required
- Require backend + provider (FCM, APNs, Expo)
- Get push token for targeting; store securely on backend

Today’s Lecture

Backend Integration

Why Mobile Apps Need Backends

Static Apps: Self-contained app

- All data is **hardcoded** in the code
- Or **stored locally** (e.g., React Native Async Storage)
- Not communicate with any external API or server
- Behave the same for every user
- Fine for solo, offline experiences, simple prototypes

But real apps are **dynamic** and **multi-user**

Mobile Apps Need Backends

Real apps are **dynamic** and **multi-user**

- **Fetch live data** from an API or backend service
- **Update** based on user input, server changes, or other users' actions
- Enable **multi-user**, **collaborative**, **real-time** experiences
- Examples: Weather app, chat app

Static vs. Dynamic Data

Aspect	Static (Local)	Dynamic (Backend)
Data Source	Async Storage, JSON files	API endpoints, databases
Update Freq.	Manual	Real-time or on-demand
Multi-User	No (device-only)	Yes (shared, synced)
Examples	Todo list from local array	Social feed from server posts

Backend Integration

Frontend: App UI, state, API calls (`fetch`/`axios`)
Backend: Validates, processes, stores/retrieves

Backend Integration Options

- **REST APIs**: Communicate with external or custom-built servers through HTTP requests
- **Backend-as-a-Service (BaaS)**: Use ready-made backend infrastructure (databases, auth, storage, functions)

REST APIs

- **Public APIs**: Ready-to-use data sources available online
- Examples: **JSONPlaceholder** (hosted, online API returns fake but realistic data), **OpenWeather**
- **Mock APIs**: Simulated backends used for testing and development without a real server
- Example: **MSW (Mock Service Worker)** (local dev tool takes network requests & returns mock responses)
- **Custom APIs**: Build your own backend with Node.js + Express, and deploy to services like **Vercel**
- Full control over logic, database, and authentication

BaaS

Backend-as-a-Service platforms provide ready-made backend infrastructure

- Databases, authentication, file storage, and more
- Minimal setup: No server configuration required
- Scales automatically and provides built-in security

  You can focus on building your app’s features rather than managing servers

BaaS: Options

Firebase: Google's full backend suite for modern apps

- **Cloud Firestore**: Scalable NoSQL cloud database
- **Data Connect**: Relational database service using a fully-managed PostgreSQL database powered by Cloud SQL
- **Cloud Storage**: Store and serve user-generated content
- **Authentication & Authorization**
- **Cloud Functions**: Automatically run backend code in response to HTTP requests or app events

Excellent for quick setup and **real-time, event-driven** apps

BaaS: Options

Firebase: Google's full backend suite for modern apps
Supabase: Open-source alternative

- Built on **PostgreSQL**
- **Auth**: Secure authentication and authorization
- **Storage**: File storage with access control
- **Edge Functions**: Server-side TypeScript functions deployed globally

  Great for developers who prefer **SQL**, **open-source tools**, and **fine-grained control**

Supabase

Open-source Backend-as-a-Service built on top of PostgreSQL

- Database + Auth + Realtime Updates + Storage in one platfrom
- Has a free tier suitable for student projects
- Scales easily to production-level deployments

Supabase: Setup

1. Sign in with GitHub: **supabase.com/dashboard/sign-in**
2. Create a new organization
   - Choose a name and select a plan
3. Create a new project
   - Give it a name and password
   - Once created, note down your **Project URL** and **Public API Key**

Secure API Keys

Never hard-code API keys directly in source files

- Unsafe and makes your project harder to share or deploy

  Store them as **environment variables**

- In the root of your Expo project, create a file named `.env`

supabase-demo/.env

```
1 SUPABASE_URL=https://your-project-id.supabase.co
2 SUPABASE_PUBLIC_KEY=your-public-api-key
```

- Never commit this file to GitHub

Expo Env Variables

Expo automatically loads environment variables with an `EXPO_PUBLIC_` prefix from `.env` files

supabase-demo/.env

```
1 EXPO_PUBLIC_SUPABASE_URL=https://your-project-id.supabase.co
2 EXPO_PUBLIC_SUPABASE_PUBLIC_KEY=your-public-api-key
```

- Supabase public API key is used by client-side apps (e.g., React Native / Expo app) to connect to Supabase
- Safe to expose in client app — it has limited permissions, controlled by **Row Level Security (RLS)**
- RLS: A database feature that restricts access to specific rows in a table based on user roles or conditions

Public vs. Service Key

Supabase **public API key** (also called **anon key**) is safe to expose in client app
Service key (also called **service_role key**) has full access to your database

- Never include it in client apps
- Use it only on the server (e.g., in Supabase Edge Functions or backend scripts)

Use Environment Variables

Once defined, you can access them anywhere in your code using `process.env`

```
const supabaseUrl = process.env.EXPO_PUBLIC_SUPABASE_URL
const supabaseKey = process.env.EXPO_PUBLIC_SUPABASE_PUBLIC_KEY
```

This keeps

- Credentials secure
- Code clean
- Setup flexible across environments (development, testing, production)

`supabase-js`

Supabase JavaScript Client Library makes it easy to integrate with Supabase from React Native or web apps

- Interact with Postgres database
- Listen for real-time updates (e.g., when rows are updated)
- Upload and manage files with Supabase Storage
- Handle authentication (sign up, sign in, session management)
- Invoke Edge Functions

Install

```
npm install @supabase/supabase-js
```

Then initialize Supabase in your app

```
import { createClient } from "@supabase/supabase-js";

const supabaseUrl = process.env.EXPO_PUBLIC_SUPABASE_URL as string;
const supabaseKey = process.env.EXPO_PUBLIC_SUPABASE_PUBLIC_KEY as string;
const supabase = createClient(supabaseUrl, supabaseKey);
```

Initialize Supabase Client

Best practice: Initialize the Supabase client once in a separate file, then reuse that instance throughout your app

supabase-demo/utils/Supabase.ts

```ts
import { createClient } from "@supabase/supabase-js";

const supabaseUrl = process.env.EXPO_PUBLIC_SUPABASE_URL as string;
const supabaseKey = process.env.EXPO_PUBLIC_SUPABASE_PUBLIC_KEY as string

export const supabase = createClient(supabaseUrl, supabaseKey);
```

- All parts of your app share the same Supabase connection
- No repeated setup logic in multiple files
- If you change project URL or key, update it in one place

Create A Table

In Supabase dashborad → Database → Create a new table

- Table Name: `tasks`
- Columns: `id` (uuid, primary), `title` (text), `completed` (bool)

TypeScript Support

`supabase-js` provides TypeScript support

- Type inference, autocompletion, type-safe queries, and more
- Detects `not null` constraints
- Nullable columns are typed as `T | null`
- Detects relationships between tables

Official Doc:
supabase.com/docs/reference/javascript/typescript-support

Generate TS Types

- Use **Supabase CLI**

```
supabase gen types typescript --project-id abcdefghijklmnopqrst > database.ty
```

- Generate from **dashboard**
- Dashboard → API Docs → TABLE AND VIEWS → Introduction → Generating types

Generate TS Types

supabase-demo/database.types.ts

```ts
export type Database = {
  public: {
    Tables: {
      tasks: {
        Row: {  // the data expected from .select()
          completed: boolean | null
          id: string
          title: string
        }
        Insert: {  // the data to be passed to .insert()
          completed?: boolean | null
          id?: string
          title: string  // `not null` columns with no default must be su
        }
        Update: {  // the data to be passed to .update()
          completed?: boolean | null
          id?: string
          title?: string  // `not null` columns are optional on .update()
        }
```

Use TS Type Definitions

Initialize Supabase client with the generated `Database` type

supabase-demo/utils/Supabase.ts

```ts
import { createClient } from "@supabase/supabase-js";
import { Database } from "../database.types";

const supabaseUrl = process.env.EXPO_PUBLIC_SUPABASE_URL as string;
const supabaseKey = process.env.EXPO_PUBLIC_SUPABASE_PUBLIC_KEY as string

export const supabase = createClient<Database>(supabaseUrl, supabaseKey);
```

Now all queries are type-safe and autocompleted

Fetch Data

Fetch all tasks from the `tasks` table using `supabase-js`

```
import { supabase } from "@/utils/Supabase";

async function fetchTasks() {
  const { data, error } = await supabase.from("tasks").select("*");

  if (error) {
    console.error("Error fetching tasks:", error);
    return [];
  }

  return data;
}
```

- `.from("tasks")`: Select the tasks table
- `.select("*")`: Fetch all columns

Use in Component

Fetch all tasks when loading the app and render them in a list

supabase-demo/App.js

```
import { useEffect, useState } from "react";
import { Text, View, FlatList } from "react-native";
import { supabase } from "./utils/Supabase";
import { Database } from "./database.types";

type Task = Database["public"]["Tables"]["tasks"]["Row"];

export default function TaskList() {
  const [tasks, setTasks] = useState<Task[]>([]);

  useEffect(() => {
    async function loadTasks() {
      const { data, error } = await supabase.from("tasks").select("*");
      if (error) console.error(error);
      else setTasks(data);
    }

    loadTasks();
  }, []);
```

Live Demo

supabase-demo/App.js

```
import { useEffect, useState } from "react";
import { Text, View, FlatList, StyleSheet } from "react-native";
import { SafeAreaView, SafeAreaProvider } from "react-native-safe-area-co
import { supabase } from "./utils/Supabase";
import { Database } from "./database.types";

type Task = Database["public"]["Tables"]["tasks"]["Row"];

export default function TaskList() {
  const [tasks, setTasks] = useState<Task[]>([]);

  useEffect(() => {
    async function loadTasks() {
      const { data, error } = await supabase.from("tasks").select("*");
      if (error) console.error(error);
      else setTasks(data);
    }

    loadTasks();
```

Insert New Task

```ts
import { supabase } from "@/utils/Supabase";
import { Database } from "@/database.types";

type TaskInsert = Database["public"]["Tables"]["tasks"]["Insert"];

async function addTask(newTask: TaskInsert) {
  const { error } = await supabase.from("tasks").insert(newTask);

  if (error) {
    console.error("Error adding task:", error);
  }
}
```

Typed input from `TaskInsert`

`.insert(newTask)`: Add a new row to the table

Use in Component

supabase-demo/App.js

```
import { useState } from "react";
import { Button, TextInput, View } from "react-native";
import { supabase } from "./utils/Supabase";
import { Database } from "./database.types";

type TaskInsert = Database["public"]["Tables"]["tasks"]["Insert"];

export default function App() {
  const [title, setTitle] = useState("");

  async function handleAddTask() {
    if (!title.trim()) return;

    const newTask: TaskInsert = {
      title,
      completed: false,
    };

    const { error } = await supabase.from("tasks").insert(newTask);
```

Live Demo

supabase-demo/App.js

```
import { useState } from "react";
import { Button, TextInput, View, StyleSheet } from "react-native";
import { supabase } from "./utils/Supabase";
import { Database } from "./database.types";

type TaskInsert = Database["public"]["Tables"]["tasks"]["Insert"];

export default function App() {
  const [title, setTitle] = useState("");

  async function handleAddTask() {
    if (!title.trim()) return;

    const newTask: TaskInsert = {
      title,
      completed: false,
    };

    const { error } = await supabase.from("tasks").insert(newTask);
```

Fetch + Insert

supabase-demo/App.js

```
import { useState, useEffect } from "react";
import { Button, FlatList, Text, TextInput, View } from "react-native";
import { supabase } from "./utils/Supabase";
import { Database } from "./database.types";

type Task = Database["public"]["Tables"]["tasks"]["Row"];
type TaskInsert = Database["public"]["Tables"]["tasks"]["Insert"];

export default function App() {
  const [tasks, setTasks] = useState<Task[]>([]);
  const [title, setTitle] = useState("");

  // Fetch tasks when component mounts
  useEffect(() => {
    async function loadTasks() {
      const { data, error } = await supabase.from("tasks").select("*");
      if (error) console.error("Error fetching tasks:", error);
      else if (data) setTasks(data);
    }
```

Live Demo

supabase-demo/App.js

```
import { useState, useEffect } from "react";
import {
  Button,
  FlatList,
  Text,
  TextInput,
  View,
  StyleSheet,
} from "react-native";
import { SafeAreaView, SafeAreaProvider } from "react-native-safe-area-co
import { supabase } from "./utils/Supabase";
import { Database } from "./database.types";

type Task = Database["public"]["Tables"]["tasks"]["Row"];
type TaskInsert = Database["public"]["Tables"]["tasks"]["Insert"];

export default function App() {
  const [tasks, setTasks] = useState<Task[]>([]);
  const [title, setTitle] = useState("");
```

Supabase: CRUD

- Insert: `supabase.from("tasks").insert(newTask)`
- Fetch: `supabase.from("tasks").select("*")`
- Update: `supabase.from("tasks").update({ title: "Updated Task"}).eq("id", 1)`
- Delete: `supabase.from("tasks").delete().eq("id", 1)`

  Filters: **supabase.com/docs/reference/javascript/using-filters**

Recap: Backend Integration

Enable dynamic, multi-user, and persistent mobile experiences

- Options
- **REST APIs**: Public, Mock, Custom
- **BaaS (Backend-as-a-Service)**: Ready-to-use database, auth, storage
  - **Firebase**: Google’s full backend suite
  - **Supabase**: Open-source alternative

Recap: Supabase

- Setup via `@supabase/supabase-js`
- Use environment variables for API keys
- Type-safe queries with generated TypeScript types
- CRUD: `insert()`, `select()`, `update().eq()`, `delete().eq()`

Today’s Lecture

Backend Integration
User Authentication

Auth

Authentication

- Verify **who you are** (identity)

Authorization

- Determine **what you can do** (permissions)

Why Authentication Matters

- **Data Protection**: Prevent unauthorized access to private or sensitive information
- **User Personalization**: Enable secure access to user-specific data (e.g., profiles, settings, saved progress)
- **Trust & Security**: Build user confidence in your app

Authentication Flow

- **Sign Up**: User creates an account (e.g., email, password)
- **Sign In**: Server verifies credentials and issues a **token**
- **Logout**: **Token/session** is cleared or invalidated

Tokens and Sessions

After successful login

- Server creates a **session** — Remembers that this user is authenticated
- Server issues a **token** — A piece of data that represents this session
- Example: **JWT (JSON Web Token)** for **stateless authentication**

Stateless Authentication

Server does not store any session data about the user between requests
Each request includes everything needed to verify the user — typically a JWT
Example:

- Server encodes user info (e.g., ID, role, expiry) and signs it
- Client sends the token with future requests
- Server verifies the token’s signature — no database lookup needed

Stateless Authentication

Analogy: Showing your passport every time you enter a border

- The officer doesn’t need to remember you
- The passport itself proves your identity

Stateful Authentication: Server keeps a session record and checks it each time

- The officer keeps a record of every traveler in a session database

Tokens and Sessions

After successful login

- Server creates a **session** and issues a **token**
- App stores token securely (e.g., in Expo SecureStore)
- Future API calls include token in the header
- `Authorization: Bearer <token>`
- This tells the server who is making the request

Simplified Flow

- **Sign Up** → User creates account → Server stores credentials (hashed password)
- **Sign In** → User enters credentials → Server verifies → Returns token
- **Authenticated Requests** → App includes token → Server validates token → Returns user-specific data
- **Logout** → App deletes stored token → User is logged out

Security Principles

- **Never store plaintext passwords**: Always hash and salt before saving
- Salt: Random data added before hashing
- **Use HTTPS**: Encrypt traffic between client and server
- **Secure token storage**: Use tools like Expo SecureStore
- **Short-lived tokens + refresh mechanism**: Limit the damage if a token is leaked

Authentication Options

Firebase Authentication: Authentication service by Google Firebase

- Supports email/password, phone, and OAuth providers (Google, Apple, GitHub, etc.)
- Handles tokens, user management, and session persistence
- Integrates easily with other Firebase tools (e.g., Firestore, Cloud Functions)

Authentication Options

Firebase Authentication: Authentication service by Google Firebase
Supabase Auth: Built-in authentication system for Supabase projects

- Supports email/password, OAuth (Google, GitHub, Apple, etc.), magic link (login link sent to email), and OTP (One-Time Password)
- Automatically manages sessions and JWTs
- Works seamlessly with Supabase Database and Storage

Authentication Options

Firebase Authentication: Authentication service by Google Firebase
Supabase Auth: Built-in authentication system for Supabase projects
Expo LocalAuthentication: On-device authentication using biometrics

- Face ID, Touch ID (iOS), Fingerprint (Android)
- Doesn’t require a backend
- Used to re-authenticate users locally (e.g., before showing sensitive info)

Supabase Auth

Supabase provides built-in **authentication** and **user management**

Supabase client also handles authentication automatically — no need for manual token storage

supabase-demo/utils/Supabase.ts

```ts
import { createClient } from "@supabase/supabase-js";

const supabaseUrl = process.env.EXPO_PUBLIC_SUPABASE_URL as string;
const supabaseKey = process.env.EXPO_PUBLIC_SUPABASE_PUBLIC_KEY as string

export const supabase = createClient(supabaseUrl, supabaseKey);
```

Sign Up

Creates a new user in Supabase Auth

```
const { data, error } = await supabase.auth.signUp({
  email: "student@example.com",
  password: "password123",
});
```

- Can trigger **email verification** if enabled
- Supabase dashboard → Authentication → Sign In / Providers → User Signups → Confirm email

Sign In

Log in an existing user with an email + password or phone + password

```
const { data, error } = await supabase.auth.signInWithPassword({
  email: "student@example.com",
  password: "password123",
});
```

- Returns a session containing a JWT access token
- This token is automatically used in later Supabase requests

Sign Out

```
1  const { error } = await supabase.auth.signOut();
```

- Clears the local session
- The user will need to log in again to make authenticated requests

Manage Session

Supabase automatically stores the user's session (JWT) securely

`getSession()`

- Retrieves the **current session** (if the user is already logged in)
- Useful when app starts — to restore user state

```
const { data } = await supabase.auth.getSession();
const user = data.session?.user; // null if not logged in
```

Listen for Auth Changes

Receive a notification every time an auth event happen `onAuthStateChange()`

- Subscribes to authentication events (sign-in, sign-out, token refresh)
- Keeps your UI in sync with the user's auth state

```
supabase.auth.onAuthStateChange((_event, session) => {
  setUser(session?.user ?? null);
});
```

Supabase Auth: Example

When you run the app

- A form appears asking for email and password
- You can sign up (new user) or sign in (existing user)
- Once logged in, it shows user info and a logout button

```
supabase-demo/
├── App.tsx
├── .env
└── utils/
    └── Supabase.ts
```

Supabase Auth: Example

supabase-demo/App.js

```
import { useEffect, useState } from "react";
import { Button, TextInput, Text, View } from "react-native";
import { supabase } from "./utils/Supabase";

type User = {
  id: string;
  email?: string;
};

export default function App() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [user, setUser] = useState<User | null>(null);

  // Load session when app starts
  useEffect(() => {
    async function loadSession() {
      const { data } = await supabase.auth.getSession();
      setUser(data.session?.user ?? null);
```

Live Demo

supabase-demo/App.js

```
import { useEffect, useState } from "react";
import { Button, TextInput, Text, View } from "react-native";
import { SafeAreaView, SafeAreaProvider } from "react-native-safe-area-co
import { supabase } from "./utils/Supabase";

type User = {
  id: string;
  email?: string;
};

export default function App() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [user, setUser] = useState<User | null>(null);

  // Load session when app starts
  useEffect(() => {
    async function loadSession() {
      const { data } = await supabase.auth.getSession();
```

Supabase Auth: Example

- `signUp` / `signInWithPassword` / `signOut`: Basic user actions
- `getSession`: Restore login on app start
- `onAuthStateChange`: Stay in sync with login/logout
- Supabase automatically manages token storage

Combine Auth with CRUD

Now that users can sign up, log in, and log out
We can combine authentication with our previous Supabase CRUD example

- Every logged-in user can have their own data (e.g., tasks, posts, notes)
- Supabase automatically provides `user.id` in the session

Combine Auth with CRUD

When a user creates a task, attach their ID

```
const { data: { user } } = await supabase.auth.getUser();

await supabase.from("tasks").insert({
  title: "Finish project",
  completed: false,
  user_id: user.id,
});
```

Query only the current user's tasks

```
const { data } = await supabase
  .from("tasks")
  .select("*")
  .eq("user_id", user.id);
```

Supabase handles auth and database access in one place

- No need a separate backend or manual token handling

Recap: Authentication

Verify who you are

- Authentication Flow
- Sign Up → Create account
- Sign In → Verify credentials, issue token/session
- Logout → Clear token/session
- Options
- Firebase Auth
- Supabase Auth
- Expo LocalAuthentication: biometrics

Recap: Supabase Auth

- `signUp`, `signInWithPassword`, `signOut`
- `getSession`: Restore session
- `onAuthStateChange`: Sync UI with auth events
- Automatic token/session management

Lecture Summary

Backend Integration

Support dynamic, multi-user, persistent apps

- Options:
- **REST APIs**: Public, Mock, Custom
- **BaaS**: Ready-to-use infrastructure (Firebase, Supabase)
- Supabase Setup: Use `@supabase/supabase-js`
- Store public API key as environment variable
- Type-safe queries with generated TypeScript types

Lecture Summary

User Authentication

- **Flow**: Sign Up → Sign In → Authenticated Requests → Logout
- **Token**: JWT for stateless authentication
- **Options**: Firebase Auth, Supabase Auth, Expo LocalAuthentication
- Combine Auth with CRUD: Supabase handles auth + database in one place
- Example: Attach `user.id` to user-specific data
  - Queries filtered by current user

Speaker notes